



Converting FileMan Files into InterSystems IRIS Classes

Version 2026.1
2026-04-20

Converting FileMan Files into InterSystems IRIS Classes

PDF generated on 2026-04-20

InterSystems IRIS Version 2026.1

Copyright © 2026 InterSystems Corporation

All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)

Tel: +1-617-621-0700

Tel: +44 (0) 844 854 2917

Email: support@InterSystems.com

Table of Contents

Converting FileMan Files into InterSystems IRIS Classes	1
1 Overview of the FileMan Mapping Utility	1
1.1 System Requirements	1
2 Specifying and Modifying the Default Settings	1
3 Mapping FileMan Files	7
4 Next Steps	8
5 Comparison of a FileMan File to an InterSystems IRIS Class Definition	8
6 Mapping Details	11
6.1 Mapping of FileMan Datatype Fields	11
6.2 Mapping of Variable Pointer Fields	13
6.3 Mapping of New-Style Cross-References	14
6.4 Mapping of Cross-References That Use DUZ(9)	14
7 Advanced Queries	14
 List of Tables	
Table 1:	12

Converting FileMan Files into InterSystems IRIS Classes

Note: This feature is available and fully supported only for existing users. InterSystems recommends against starting new projects with FileMan.

InterSystems IRIS® data platform provides a utility to convert applications based on FileMan files into InterSystems IRIS classes, thus providing object and SQL access to the data. Specifically, this utility (the FileMan Mapping Utility) generates InterSystems IRIS class definitions that map the data. This page assumes that the FileMan globals (^DD and ^DIC) are already loaded into the desired namespace in your system.

1 Overview of the FileMan Mapping Utility

FileMan enables you to quickly and easily create custom class mappings for your FileMan files. The classes created by the mapping utility enable you to access the file data via object access and via SQL. You can map just one, many, or all FileMan files in a namespace. For each file, you can control details such as the following:

- Which fields are mapped
- Whether the classes are read-only or have write access
- Formats for the names of the generated classes and properties (tables and fields)
- The superclass list for the generated classes
- Whether word-processing fields are mapped as list collections or as child tables

1.1 System Requirements

You need a working FileMan installation.

2 Specifying and Modifying the Default Settings

FileMan uses many settings that control how it generates classes. It is worthwhile to review these settings, modify them to values that you use most of the time, and save these as the default settings.

To specify or modify the default settings, use a command like the following:

```
Set ^%SYS("sql","fm2class",setting) = value
```

Where *setting* is an internal setting name as shown in the following list, and *value* is the value to assign to it.

When you use the methods in **\$SYSTEM.OBJ.FM2Class**, you can pass in an array of the following format:

```
arrayName(setting) = value
```

For example:

```
set fmsettings("display")=1
set fmsettings("package")="VISTA"
set fmsettings("superClasses")="%XML.Adaptor"
set fmsettings("wpIsList")= 1
```

The available settings are as follows:

childTableNameFormat

Specifies the format of the generated child table names. For this setting, specify a string that uses the following keywords along with any characters that are valid to use as table names:

- <FILENAME> — Replaced with the name of the FileMan file
- <FILENUMBER> — Replaced with the file number
- <PARFILENAME> — Replaced with the name of the parent file
- <PARFILENUMBER> — Replaced with the file number of the parent file

If you use <FILENAME> or <PARFILENUMBER>, any decimal place characters in the file number are converted to underscore characters.

Some examples of this setting:

- SUB_<FILENAME> — In this example, the table name of a child table is the string SUB_ followed by the name of the file. For example: SUB_ACCESSIBLE_FILE
- f<PARFILENUMBER>c<FILENUMBER> — In this example, the table name of a child table is the string f, followed by the number of the parent file, followed by c, followed by the number of this file. For example: f200c200_032
- <FILENAME> — In this example, the child table name is simply the same as the filename.

ClassParameters

Specifies class parameters and values for an OS object, in JSON format.

compile

Specifies whether to compile the classes after creating them. Permitted values are as follows:

- 1 — Compile the classes.
- 0 — Do not compile the classes.

compileQSpec

Specifies any class compiler qualifiers and/or flags.

dateType

Specifies the data type to use when mapping FileMan DATE fields. The default is %Library.FilemanDate.

datetimeType

Specifies the data type to use when mapping FileMan DATE/TIME fields. The default is %Library.FilemanTimeStamp.

defineVASITE

Creates a new field VA_SITE in the table, to store the Station ID of the VA site. This field can be useful for telling the source of the data when joining data from multiple VA or FileMan servers. Permitted values are as follows:

- 1 — Create the VA_SITE field.
- 0 — Do not create the VA_SITE field.

deleteQSpec

Specifies any class deletion qualifiers and/or flags.

display

Controls how the results are displayed. Permitted values are as follows:

- 0 — No display.
- 1 — Minimal display.
- 2 — Full display. This is the default.

expandPointers

Specifies whether the utility creates additional computed properties to expand the pointer field. Permitted values are as follows:

- 0 — The utility does not add any computed properties to expand the pointer field.
- 1 — The utility adds a computed property that expands the pointer field and is equal to the NAME (.01) field in the referenced file.
- 2 — The utility adds a computed property that expands the pointer field and is equal to the NAME (.01) field in the referenced file. The naming conventions used for this option is the pointer field will be named after the referenced file with “_ID” appended to the field name.
- 3 — This option is the same as 2 except that the name of the reference field will be <Field>_f<pointed-to-file-id#>ID.

expandSetOfCodes

Specifies whether to define an expanded SOC (SetOfCodes) table for each Set Of Codes field that is mapped. Permitted values are as follows:

- 1 — For each Set of Codes field that is mapped to a class, the utility will generate a read-only class/table that maps to the CODE and MEANING of the Set Of Codes. The name of the table is <tablename>_SOC_<field-name>. The SOC table has two fields. CODE maps to the name of the CODE and MEANING is the external meaning for the CODE. This table is mapped directly to the ^DD global, so any updates to the definition in the ^DD global will be immediately reflected in the data returned by the table.
- 2 — These SOC class/tables will not be created. This is the default.

extendedMapping

Specifies a string to be inserted into the global name in the map definitions for extended mapping purposes. For example, you might specify ["SD"] and your global is mapped as ^["SD"]LR(. . .) instead of ^LR(. . .).

This can be any valid string that can be used for extended global mapping and must include the [. . .] or | . . . | brackets.

fieldNameFormat

Specifies how the utility will generate the SQL field name. Permitted values are as follows:

- **Exact** — The SQL field name will use exactly the same case as the FileMan field name. For example, FileMan field 'DEA EXPIRATION DATE' becomes SQL field name 'DEA_EXPIRATION_DATE'.
- **Upper** — Letters in the FileMan field name are folded to upper case to generate the SQL field name. For example, FileMan field 'DEA expiration date' becomes SQL field name 'DEA_EXPIRATION_DATE'.
- **Lower** — Letters in the FileMan field name are folded to lower case to generate the SQL field name. For example, FileMan field 'DEA EXPIRATION DATE' becomes SQL field name 'dea_expiration_date'.
- **Pascal** — The first letter in the identifier and the first letter of each subsequent concatenated word are capitalized. Also, spaces and underscored are removed. For example, FileMan field 'DEA EXPIRATION DATE' becomes the SQL field name 'DeaExpirationDate'.
- **Camel** — The first letter of an identifier is lowercase and the first letter of each subsequent concatenated word is capitalized. Also, spaces and underscored are removed. For example, FileMan field 'DEA EXPIRATION DATE' becomes the SQL field name 'deaExpirationDate'.

iенFieldName

Specifies the name of the IEN field, which is **IEN** by default.

JSONAdaptor

Enables JSON Adapter functionality for use with mapped classes. The value of **JSONAdaptor** is a JSON-format string of class parameters and values. Permitted parameters are as follows:

- **%JSONENABLED**
- **%JSONIGNOREVALIDFIELD**
- **%JSONNULL**
- **%JSONIGNORENULL**
- **%JSONREFERENCE**
- **ComputedAlwaysJSONINCLUDE**

For details about **JSONAdaptor** parameters, see **%JSON.Adaptor** in the class reference.

ComputedAlwaysJSONINCLUDE allows specification of the **%JSONINCLUDE** property parameter for properties defined as **Calculated** and **SqlComputed**.

For best results, the following settings are recommended:

- Set **%JSONREFERENCE** to **ID**.
- Set **ComputedAlwaysJSONINCLUDE** to **OUTPUTONLY**.

logFile

Specifies the name of the file into which the utility should log output. Type a filename.

mapMUMPSCrossReferences

Controls whether MUMPS cross-references are mapped as index maps. Use one of the following values:

- **0** (default) means the MUMPS cross-references will not be defined as index maps in the class.

- 1 means that a MUMPS cross-reference will be mapped as an index in the class if possible and as long as this is not an "A" cross reference. "A" cross-references are defined for sorting, and not fully supported for lookups. The cross-references SET code still needs to be in the form `S ^ . . . = ""` in order for the index map to be defined in the class.
- 2 means that a MUMPS cross-reference will be mapped as an index in the class if possible, including "A" cross references. The cross-references SET code still needs to be in the form `S ^ . . . = ""` in order for the index map to be defined in the class.

WARNING! Using options 1 or 2 might result in an index defined in the class/table that is for a partial index, which means it may not include all records in the table/file, and queries could potentially produce incorrect results. This setting should only be used on files where you know the MUMPS cross references are full indexes.

nameLength

Specifies the maximum length of property names, foreign key names, and trigger names produced by this utility. The default is 180, which corresponds to an increase in the length of InterSystems IRIS class member names in a recent release. Use this option if you want to keep the names of these items at the previous shorter maximum (31 characters).

owner

Specifies the username to use as the owner of the classes created. The default is the current value in `$Username`.

package

Specifies the name of the package to create the classes in.

readonly

Specifies whether the generated classes should be read-only. Permitted values are as follows:

- 1 — Generate read only classes.
- 0 — Generate read/write classes.

recursion

Controls whether sub-files and pointer are also mapped. Permitted values are as follows:

- 0 — No recursion. Only this file is mapped. No sub-files or pointers are mapped.
- 1 — Partial recursion. The file is mapped, along with one level of sub-files and pointers.
- 2 — Full recursion. The file is mapped, along with all sub-files and pointers. This is the default.
- DEPTH=#, where # is an integer— This option enables you to specify an exact depth for the recursion. For example, DEPTH=2 means that two levels of recursion for sub-files and pointers are mapped.

requiredType

Controls which required FileMan fields are defined as required properties. Permitted values are as follows:

- 1 — FileMan fields marked as required are defined as required properties. This is the default.
- 0 — Only FileMan fields defined as Required Identifiers are defined as required properties.

retainClass

Specifies whether to recreate the entire class if it already exists. Permitted values are as follows:

- 0 — The utility deletes and recreates the class, which means that SQL privileges and any add-ons to the class are lost.
 - 1 — The utility recreates the properties, storage, indexes, foreign keys, and so on, rather than the entire class.
- Notes:

- Note:**
- After mapping a class, if you move the class to another namespace or another system, and attempt to map it again from the new location with the `RetainClass` set to 1, you must also manually move/copy the `^oddFMD` global too, because this global stores the metadata required by the `RetainClass` setting.
 - The `RetainClass` setting is not meant to work with SOC classes produced when you set `expandSetOfCodes` to 1; that is, if you add your own parameters, properties, indexes, and so on to these classes, those changes are not retained, even if `RetainClass` is set to 1.

setOfCodesEnum

Specifies the data type to use when mapping a Set Of Codes field. The options are `%Library.EnumString` (the default) and `%Library.String`.

Using `%Library.EnumString` provides an advantage because it provides the **`OdbcToLogical()`** and **`LogicalToOdbc()`** methods, which allow you to use the meaning of the Set Of Codes, rather than the code value from xDBC client applications.

strictData

Specifies whether the generated classes include `STRICTDATA=1` in the definitions of any properties of type `%Library.FilemanDate` and `%Library.FilemanTimeStamp`.

This `STRICTDATA` parameter affects the **`LogicalToOdbc()`** and **`LogicalToDisplay()`** methods for these data types. When `STRICTDATA=0`, the default, the methods will contain the same code they did previously. When `STRICTDATA=1`, the code in the **`LogicalToFormat()`** methods that handles invalid Logical date and time values is removed. Use `STRICTDATA=1` if your database contains invalid or incomplete date or time values, and you do not want assumptions made about what the correct data should be.

superClasses

Specifies the superclass list for each of the mapped classes. Specify a string with a comma-separated list of class names.

tableNameFormat

Specifies the format of the generated table name. For this setting, specify a string that uses the following keywords along with any characters that are valid to use as table names:

- `<FILENAME>` — Replaced with the name of the FileMan file
- `<FILENUMBER>` — Replaced with the file number

If you use `<FILENAME>`, any decimal place characters in the file number are converted to underscore characters.

See the examples for `childTableNameFormat`.

variablePointerValueField

Specifies how to handle variable pointer fields. Permitted values are as follows:

- 1 — For each variable pointer field mapped, FM2Class will also create a computed property that expands the variable pointer field value and is equal to the .01 field in the referenced file. This field will be named `<Variable_Pointer_Field_VALUE>`.

For example, an FM variable pointer field WHO is defined to point to either the EMPLOYEE file or the PATIENT file. By default, FM2Class creates two fields, one that points to the EMPLOYEE file and another one that points to the PATIENT file. Only one of these fields will be non-NULL. If WHO is an EMPLOYEE, the WHO_EMPLOYEE field will contain the ID/IEN of the EMPLOYEE. If WHO is a PATIENT, WHO_PATIENT will contain the ID/IEN of the PATIENT.

FM2Class *also* creates a third field, WHO_VALUE, which computes to the .01 field of the EMPLOYEE record if the WHO points to an EMPLOYEE, or the .01 field of the PATIENT record if WHO points to a PATIENT.

- 0 — FM2Class does not create this third field.

wholeFileScreen

Specifies whether to consider the Whole File Screen feature when generating classes. This FileMan feature allows you to specify a condition that controls whether a given record is returned. Specifically if the condition evaluates to 1, the record is returned; otherwise, the record is not returned.

If `wholeFileScreen` is 1, and if a file uses the Whole File Screen feature, then the generated class includes the code needed to filter the screened rows when querying the associated table through SQL.

The `wholeFileScreen` condition can reference variables Y (the IEN value), DA (the IEN value), DA(#) (IEN values for parent IDs, X (the value of the .01 (NAME) field), and U ("^").

For `wholeFileScreen`, the default is 0.

wpIsList

Specifies how to map word-processing fields. Permitted values are as follows:

- 0 — Convert as child tables.
- 1 — Convert as list collections.
- 2 — Convert as both list collections and child tables.
- 3 — Convert as a single long string field.

3 Mapping FileMan Files

Use methods in `%SYSTEM.OBJ.FM2Class`, as follows:

- To map all FileMan files in the current namespace:

ObjectScript

```
Do $SYSTEM.OBJ.FM2Class.All(.fmSettings, .classCount)
```

- To map one FileMan file:

ObjectScript

```
Do $SYSTEM.OBJ.FM2Class.One(fileName, .fmSettings, .fmFields, .classCount)
```

- To map some of FileMan files in the current namespace:

ObjectScript

```
Do $SYSTEM.OBJ.FM2Class.Some(fileList, .fmSettings, .fmFields, .classCount)
```

The arguments are as follows:

- *fmSettings* is an optional array passed by reference with any settings you would like the utility to use, as described earlier in this section.
- *classCount* is optional and is also passed by reference. It returns the number of classes created by the mapper utility.
- *fileName* is the FileMan file number of the file you want to map.
- *fmFields*, if defined, limits the fields in the file that are mapped. This is an array of the form `fmFields(file-number, field-number)`. Any required fields and fields defined in this array are mapped in the class definition. If this array is empty or not defined, all fields in the file are mapped. This array is passed by reference.
- *fileList* is the FileMan file numbers of the files you want to map. Specify a comma-delimited list of file number or ranges of file numbers. Or specify an array of file numbers passed by reference.

The class %SYSTEM.OBJ.FM2Class also provides the methods **Version()** and **GetVersion()**.

4 Next Steps

After using the conversion utility, you should verify that you have SQL access to the generated classes. To do so, use Management Portal. At a minimum, check that you can do the following:

- Browse the schema.
- Execute SQL queries against the new classes.

It may also be necessary to create global and routine mappings in the namespace. Check the routines used in your generated classes to be sure that all of them are available in this namespace.

5 Comparison of a FileMan File to an InterSystems IRIS Class Definition

To orient yourself, you may find it helpful to compare a simple FileMan file to the resulting InterSystems IRIS class definitions.

Consider the simple example of a file called `PostalCode`. If we look at the file attributes for this file (via the FileMan Data Dictionary Utility), we see the cross-references and indexes for this file. For example:

```

STANDARD DATA DICTIONARY #5.12 -- POSTAL CODE FILE
FEB 20,2009@14:52:06 PAGE 2
STORED IN ^XIP(5.12, *** NO DATA STORED YET *** SITE: VA PLATINUM ACCOUNT U
CI: TSTSQLOBJ,VISTA (VERSION 8.0)

DATA      NAME      GLOBAL      DATA
ELEMENT   TITLE      LOCATION   TYPE
-----
"W1": D EN^DDIOL($P(^ (0),U,2)_" " _$P(^ (0),U,6)_" " _$P(^ (0),U,7),"", "
?0")

CROSS
REFERENCED BY: COUNTY(AC), MAIL CODE(B), CITY(C), CITY KEY(D),
               UNIQUE KEY (VA) (E)

INDEXED BY:    MAIL CODE & CITY & COUNTY & STATE & INACTIVE DATE & CITY KEY &
               PREFERRED CITY KEY & CITY ABBREVIATION & UNIQUE KEY (VA) (AD)

5.12,.01      MAIL CODE      0;1 FREE TEXT (Required)

```

The same utility also shows us the definitions of the fields in this file. In this case, the titles of the fields are as follows (when listed alphabetically)

- City
- City Abbreviation
- City Key
- County
- Inactive Date
- Mail Code
- Preferred City Key
- State
- Unique Key (VA)

The FileMan Data Dictionary Utility also shows that the PostalCode file includes two pointers:

- A pointer to the County Code file
- A pointer to the State file

If we map this file, five classes are created. If we had specified TEST as the package name, the five classes would be as follows:

- TEST.COUNTY
- TEST.COUNTYCODE
- TEST.POSTALCODE
- TEST.STATE
- TEST.ZIPCODE

The class definition for TEST.POSTALCODE is too long to show, but the details are summarized here:

- The DATECREATED class parameter is set equal to the time when the class was created or updated.

- The *FILEMANFILENAME* parameter is set equal to "POSTAL CODE" and the *FILEMANFILENUMBER* parameter is set equal to 5.12. These two parameters indicate the FileMan file from which this class was mapped.
- The class includes the following properties:
 - CITY
 - CITYABBREVIATION
 - CITYKEY
 - COUNTY
 - IEN
 - INACTIVEDATE
 - MAILCODE
 - PREFERRED CITYKEY
 - STATE
 - UNIQUEKEYVA

The definitions of these properties are derived from the definitions of the corresponding fields. Also note that the FileMan Internal Entry Number is represented explicitly as a property.

Descriptive text is carried over and is used as comments. For example:

```
/// FileMan Field Label: 'STATE' FileMan Field Number: '3'  
/// This field contains a pointer to the State File to represent the state  
/// associated with this Postal Code.  
Property STATE As TEST.STATE [ SqlColumnNumber = 6, SqlFieldName = STATE ];
```

Several of these properties (like this one) are references to the other generated classes. This representation addresses the cross-references defined in the *PostalCode* file.

- This utility creates foreign key constraints for each pointer field created. The foreign key constraint simply references the ID of the referenced table.

In this example, the class includes the following foreign keys to represent the pointer references in the *PostalCode* file:

- **FKeyCOUNTY(COUNTY)**, which references TEST.COUNTYCODE()
- **FKeySTATE(STATE)**, which references TEST.STATE()

Also see [Mapping of Variable Pointer Fields](#).

- The class includes one index:

```
Index IDKeyIndex On IEN
```

- Finally, the class includes three triggers that act when filing is performed through the FileMan filer:
 - **BeforeDeleteFiling**
 - **BeforeInsertFiling**
 - **BeforeUpdateFiling**

6 Mapping Details

This section contains notes about how the methods in `$$SYSTEM.OBJ.FM2Class` generate InterSystems IRIS class definitions.

- [How the utility maps FileMan datatype fields](#)
- [How the utility maps variable pointer fields](#)
- [How the utility maps new-style cross-references](#)
- [How the utility maps cross-references that use DUZ\(9\)](#)

6.1 Mapping of FileMan Datatype Fields

The FileMan mapping utility supports conversion of certain field types, as follows:

- BOOLEAN
- LABEL REFERENCE
- TIME
- YEAR
- UNIVERSAL TIME
- FT POINTER
- FT DATE
- RATIO

6.1.1 Supporting InterSystems IRIS Datatype Classes

The following InterSystems IRIS datatype classes exist to enable support of certain field types:

- `%Library.FilemanTime`
- `%Library.FilemanYear`
- `%Library.FilemanTimeStampUTC`

Note: The conversion methods of `%Library.FilemanTimeStampUTC` assume a full FileMan 22.2 runtime environment is installed, and that the `DUZ(2)` variable is defined and references to `INSTITUTION` that includes defined `COUNTRY` and `LOCATION TIMEZONE` values.

The main purpose of these datatype classes is to provide *internal* (logical) and *external* (display/ODBC) values for the type. They are not set up with the SQL engine to be compatible with other date/time/timestamp types. For example, `%Library.FilemanTime` will not compare properly with a `%Time` type in a query, because the logical values of the two types are different. The logical value for these three types are strings, and they use string collation to force the compared value to be a string.

If you want to do something like compare a `%Library.FilemanTime` field with the current time, you must first convert the FileMan time field to a `%Time` format so the comparison is valid. An example of how to do this is:

```
SELECT ... FROM ... WHERE CAST(%external(FM_TIME_FIELD) AS TIME) > CURRENT_TIME
```

6.1.2 Supported FileMan Field Types

BOOLEAN Fields

A field defined as a BOOLEAN data type can have only two entry choices:

Table 1:

<i>Internal</i>	<i>External</i>
1	YES
0	NO

LABEL REFERENCE Fields

A field defined as a LABEL REFERENCE data type is designed to store a tag and routine entry of the format, `<TAG>^<ROUTINE>`. It is stored as a free-text field.

LABEL REFERENCE fields map to InterSystems IRIS as `%Library.String`.

TIME Fields

A field defined as a TIME data type can accept many of the date/time entries, but only stores the time portion. For example:

- *Internal* — 150943
- *External* — 15:09:43

TIME fields map to InterSystems IRIS as `%Library.FilemanTime`.

YEAR Fields

A field defined as a YEAR data type can accept many of the date entries, but only stores the year portion. For example:

- *Internal* — 3160000
- *External* — 2016

YEAR fields map to InterSystems IRIS as `%Library.FilemanYear`.

UNIVERSAL TIME Fields

A field defined as a UNIVERSAL TIME data type can accept many of the date/time entries. It stores the data/time in a format with the local time, including an indicator showing the offset from Universal Time. The first 14 characters of the internal storage of the UNIVERSAL TIME data type are exactly like the current DATE/TIME data type that includes seconds. The three characters in positions 15, 16, and 17 indicate the UTC time offset in 5-minute increments. For example:

- *Internal* — 3160106.080336440
- *External* — JAN 6,2016@08:03:36 (UTC-5:00)

In this example, $(440-500)/12=-5$. This is a negative five hour offset from UTC.

UNIVERSAL TIME fields map to InterSystems IRIS as `%Library.FilemanTimestampUTC`.

FT POINTER Fields

A field defined as a FT POINTER data type works like the POINTER data type, but internally stores the free text that was returned from the pointed-to value. For example:

- *Internal* — PATCH,USER
- *External* — PATCH,USER

FT POINTER fields map to InterSystems IRIS as %Library.String.

FT DATE Fields

A field defined as a FT DATE data type works like the DATE/TIME data type, but internally stores the free text that was input by the user to determine the date. For example:

- *Internal* — T-1
- *External* — T-1

FT DATE fields map to InterSystems IRIS as %Library.String.

RATIO Fields

A field defined as a RATIO data type is designed to accept two numbers with a colon character (:) between the two numbers. It is formatted and stored like a mathematical ration. For example:

- *Internal* — 1:14
- *External* — 1:14

RATIO fields map to InterSystems IRIS as %Library.String.

6.2 Mapping of Variable Pointer Fields

Each variable pointer field in the file that is mapped to a InterSystems IRIS class defines a property in the class that is marked with the `SqlComputed` keyword. For each file to which this variable pointer field refers, an additional property is created in the class, and this property is marked with the `SqlComputed` and `Calculated` keywords. If *VariablePointerFieldName* is the name of the variable pointer field, and *PointerFileName* is the name of the file to which it points, the added property is *VariablePointerFieldNamePointerFileName*; the SQL field name for this property is *VariablePointerFieldName_PointerFileName*.

For example, suppose the file ABC includes a variable pointer field called VP. The VP field can point to the Red, White, or Blue files. This creates the following properties in the class definition:

Property	SqlFieldName	Details
VP		SQL computed upon INSERT and UPDATE. Stored in the map definition. Contains the internal value for the variable pointer field. Usually something like "41;DIC(40.7," which means it points to row 41 in the 40.7 file.
VPRed	VP_Red	A computed field that is null unless the VP field for this row points to the Red file. To update the value of VP via SQL INSERT or UPDATE to point to a row in the Red table/file, set the value of VP_Red to the ID of that row.
VPWhite	VP_White	A computed field that is null unless the VP field for this row points to the white file. To update the value of VP via SQL INSERT or UPDATE to point to a row in the white table/file, set the value of VP_White to the ID of that row.
VPBlue	VP_Blue	A computed field that is null that is NULL unless the VP field for this row points to the Blue file. To update the value of VP via SQL INSERT or UPDATE to point to a row in the Blue table/file, set the value of VP_Blue to the ID of that row.

In all cases, the VP field is computed and stored automatically.

In addition to defining the three reference fields VP_Red, VP_White, and VP_Blue, the utility also creates three foreign key constraints in the ABC class for these references. This is done to ensure referential integrity is maintained between the files.

6.3 Mapping of New-Style Cross-References

This utility converts new-style cross-references to index maps if the cross-reference is defined by simple set/kill logic. This allows the InterSystems IRIS query optimizer to choose a new-style cross-reference index (if one exists) and increase query performance in some cases.

6.4 Mapping of Cross-References That Use DUZ(9)

FM2Class maps a cross-reference (original or new style) that was previously not mapped because the cross-reference subscripts included a reference to the variable DUZ(9) or the subscript had the form +\$G(varname), where *varname* represents any variable reference.

If you have such cross-reference definitions, make sure that the DUZ(9) variable, or possibly any variable referenced by +\$G(varname) is defined in any process running SQL queries against tables with indexes mapped from these cross-references. To set DUZ(9) for a process connecting via xDBC, use \$SYSTEM.SQL.SetServerInitCode().

7 Advanced Queries

You can use the classes %FileMan.File and %FileMan.Field to query the FileMan data dictionary.

Also, you can use the classes %FileMan.MappedFile and %FileMan.MappedField to view metadata regarding the mapping between FileMan and the class definition.

For information on the %FileMan classes, see the InterSystems Class Reference.